```
'use strict';
/*
  Night Drive 3D - multiplayer server
  - Zero npm dependencies (own RFC6455 WebSocket implementation). Needs Node 16+.
  - ALSO hosts accounts, invites, cloud save and Owner/Admin permissions under /api
(same port, same process).
  - Serves the game (index.html) and the WebSocket endpoint /ws on the SAME port.
  - Rooms with 6-digit codes, 2-8 players, ready system, countdown, checkpoints,
    finish order, results and rewards.
  - The server owns the room / race state. Clients only send their own car state,
    and every state update is validated (range, speed, teleport, false start).
*/
const http = require('http');
const crypto = require('crypto');
const fs = require('fs');
const path = require('path');

const PORT = +process.env.PORT || 3000;
const HOST = process.env.HOST || '0.0.0.0';
const GAME_FILE = process.env.GAME_FILE || path.join(__dirname, 'index.html');
const ALLOWED_ORIGINS = (process.env.ALLOWED_ORIGINS || '').split(',').map(s =>
s.trim()).filter(Boolean); // empty = any
const GRACE_MS = +process.env.GRACE_MS || 30000;       // reconnect window
const TICK_MS = 1000 / 12;                              // snapshot rate
const WS_GUID = '258EAFA5-E914-47DA-95CA-C5AB0DC85B11';
const VMAX = 140;                                       // absolute max plausible
speed (units/s)
const RACE_GOAL = +process.env.RACE_GOAL || 0;          // optional override
(testing)

const MODES = {
  roam:       { name: 'Free Roam',        goal: 0,    cp: 0,    min: 1, dur: 0 },
  race:       { name: 'Multiplayer Race', goal: 3000, cp: 1000, min: 2, dur: 0 },
  checkpoint: { name: 'Checkpoint Race',  goal: 4000, cp: 400,  min: 2, dur: 0 },
  stunt:      { name: 'Stunt Challenge',  goal: 0,    cp: 0,    min: 2, dur: 90 },
  trial:      { name: 'Time Trial',       goal: 3000, cp: 1000, min: 1, dur: 0 }
};
function modeCfg(m) {
  const c = Object.assign({}, MODES[m]);
  if (RACE_GOAL && c.goal) c.goal = RACE_GOAL;
  return c;
}
const XP_BY_PLACE = [300, 220, 160, 120, 90, 70, 50, 40];

/* ---------------------------------------------------------------- WebSocket */
class WS {
```

```
  constructor(sock) {
    this.sock = sock; this.buf = Buffer.alloc(0); this.open = true; this.frag = null;
    this.onmessage = null; this.onclose = null; this.lastSeen = Date.now();
this.bucket = { t: Date.now(), n: 0 };
    sock.setNoDelay(true);
    sock.on('data', d => { this.buf = Buffer.concat([this.buf, d]); if
(this.buf.length > 262144) return this.close(1009); this._parse(); });
    sock.on('close', () => this._closed());
    sock.on('error', () => this._closed());
  }
  _parse() {
    for (;;) {
      const b = this.buf; if (b.length < 2) return;
      const fin = !!(b[0] & 128), op = b[0] & 15, masked = !!(b[1] & 128);
      let len = b[1] & 127, off = 2;
      if (len === 126) { if (b.length < 4) return; len = b.readUInt16BE(2); off = 4;
}
      else if (len === 127) { if (b.length < 10) return; if (b.readUInt32BE(2))
return this.close(1009); len = b.readUInt32BE(6); off = 10; }
      if (!masked) return this.close(1002);
      if (len > 65536) return this.close(1009);
      if (b.length < off + 4 + len) return;
      const mask = b.slice(off, off + 4), payload = Buffer.from(b.slice(off + 4, off
+ 4 + len));
      for (let i = 0; i < payload.length; i++) payload[i] ^= mask[i & 3];
      this.buf = b.slice(off + 4 + len);
      this.lastSeen = Date.now();
      if (op === 8) { this._frame(8, payload.slice(0, 2)); return this.close(1000,
true); }
      if (op === 9) { this._frame(10, payload); continue; }
      if (op === 10) continue;
      if (op === 1 || op === 2 || op === 0) {
        if (op !== 0) this.frag = { op, parts: [] };
        if (!this.frag) return this.close(1002);
        this.frag.parts.push(payload);
        if (this.frag.parts.reduce((s, p) => s + p.length, 0) > 65536) return
this.close(1009);
        if (fin) {
          const msg = Buffer.concat(this.frag.parts); const isText = this.frag.op ===
1; this.frag = null;
          if (isText && this.onmessage) { try { this.onmessage(msg.toString('utf8'));
} catch (e) { console.error('handler error', e); } }
        }
      } else return this.close(1002);
    }
  }
  _frame(op, payload) {
    if (!this.open && op !== 8) return false;
    const len = payload.length; let h;
    if (len < 126) h = Buffer.from([128 | op, len]);
```

```
    else if (len < 65536) { h = Buffer.alloc(4); h[0] = 128 | op; h[1] = 126;
h.writeUInt16BE(len, 2); }
    else { h = Buffer.alloc(10); h[0] = 128 | op; h[1] = 127; h.writeUInt32BE(0, 2);
h.writeUInt32BE(len, 6); }
    try { return this.sock.write(Buffer.concat([h, payload])); } catch (e) { return
false; }
  }
  send(str) { return this.open && this._frame(1, Buffer.from(str, 'utf8')); }
  ping() { this._frame(9, Buffer.alloc(0)); }
  close(code, echoed) {
    if (!this.open) return;
    if (!echoed) { const p = Buffer.alloc(2); p.writeUInt16BE(code || 1000);
this._frame(8, p); }
    this.open = false;
    try { this.sock.end(); } catch (e) { /* ignore */ }
    setTimeout(() => { try { this.sock.destroy(); } catch (e) { /* ignore */ } },
500);
    this._closed();
  }
  _closed() { if (this._done) return; this._done = true; this.open = false; if
(this.onclose) this.onclose(); }
}

/* ---------------------------------------------------------------- ACCOUNTS /
INVITES / OWNER (same process, same port, no npm packages) */
const REQUIRE_ACCOUNT = process.env.REQUIRE_ACCOUNT === '1';
const DATA_DIR = process.env.DATA_DIR || path.join(__dirname, 'data');
const DB_FILE = path.join(DATA_DIR, 'accounts.json');
try { fs.mkdirSync(DATA_DIR, { recursive: true }); } catch (e) {
console.error('cannot create data dir', e.message); }
let DB = { users: [], saves: {}, invites: [], friends: [], audit: [], seq: 1, secret:
'', setup: '' };
try { DB = Object.assign(DB, JSON.parse(fs.readFileSync(DB_FILE, 'utf8'))); } catch
(e) { /* first start */ }
if (!DB.secret) DB.secret = crypto.randomBytes(32).toString('hex');
const SECRET = process.env.JWT_SECRET || DB.secret;                       // optional
override; otherwise generated once and stored
if (!DB.users.some(u => u.role === 'owner') && !process.env.OWNER_SETUP_CODE &&
!DB.setup) DB.setup = crypto.randomBytes(6).toString('hex');
const hasOwner = () => DB.users.some(u => u.role === 'owner');
let dbTimer = null;
function dbSave() { clearTimeout(dbTimer); dbTimer = setTimeout(() => { try {
fs.writeFileSync(DB_FILE + '.tmp', JSON.stringify(DB)); fs.renameSync(DB_FILE +
'.tmp', DB_FILE); } catch (e) { console.error('db save failed', e.message); } },
400); }
dbSave();
const b64 = b => Buffer.from(b).toString('base64url');
function hashPw(pw) { const s = crypto.randomBytes(16); return s.toString('hex') +
':' + crypto.scryptSync(pw, s, 32).toString('hex'); }
function checkPw(pw, st) { const [s, h] = String(st).split(':'); if (!s || !h) return
```

```
false; const a = crypto.scryptSync(pw, Buffer.from(s, 'hex'), 32), b = Buffer.from(h,
'hex'); return a.length === b.length && crypto.timingSafeEqual(a, b); }
function signTok(u) { const p = b64(JSON.stringify({ sub: u.id, tv: u.tv, exp:
Date.now() + 30 * 864e5 })); return p + '.' + crypto.createHmac('sha256',
SECRET).update(p).digest('base64url'); }
/* every request re-reads the account, so role changes, blocks, logout and revocation
apply at once */
function acctFromToken(t) {
  if (typeof t !== 'string') return null; const [p, s] = t.split('.'); if (!p || !s)
return null;
  const good = crypto.createHmac('sha256', SECRET).update(p).digest('base64url'); if
(s.length !== good.length || !crypto.timingSafeEqual(Buffer.from(s),
Buffer.from(good))) return null;
  try { const d = JSON.parse(Buffer.from(p, 'base64url').toString()); const u =
DB.users.find(x => x.id === d.sub); return u && u.tv === d.tv && d.exp > Date.now() ?
u : null; } catch (e) { return null; }
}
const audit = (actor, target, action, detail) => { DB.audit.push({ at: Date.now(),
actor, target, action, detail: detail || '' }); if (DB.audit.length > 500)
DB.audit.shift(); };
const isOnline = u => (u.seen || 0) > Date.now() - 70000 ||
[...sessions.values()].some(s => s.acct === u.id && s.connected);
const viewU = u => ({ playerId: u.pid, username: u.username, avatar: u.avatar, role:
u.role, blocked: !!u.blocked, perms: { ownerMode: !!u.ownerMode && !u.blocked } });
const okName = s => typeof s === 'string' && /^[A-Za-z0-9_]{3,20}$/.test(s), okPass =
s => typeof s === 'string' && s.length >= 8 && s.length <= 100;
const hits = new Map();
function limited(ip, key, max, win) { const k = key + ip, now = Date.now(); let h =
hits.get(k); if (!h || now > h.r) h = { r: now + win, n: 0 }; h.n++; hits.set(k, h);
return h.n > max; }
setInterval(() => { const n = Date.now(); hits.forEach((h, k) => { if (n > h.r)
hits.delete(k); }); }, 60000);
function readJson(req) { return new Promise((ok, no) => { let n = 0; const c = [];
req.on('data', d => { n += d.length; if (n > 1200000) { no(new Error('too large'));
req.destroy(); } else c.push(d); }); req.on('end', () => { try { ok(c.length ?
JSON.parse(Buffer.concat(c).toString()) : {}); } catch (e) { no(new Error('bad
json')); } }); req.on('error', no); }); }
const E = (code, msg) => Object.assign(new Error(msg), { code });

async function api(req, res, url) {
  const origin = req.headers.origin || '';
  const cors = { 'Access-Control-Allow-Origin': ALLOWED_ORIGINS.length ?
(ALLOWED_ORIGINS.includes(origin) ? origin : 'null') : '*', 'Access-Control-Allow-
Headers': 'Content-Type, Authorization', 'Access-Control-Allow-Methods':
'GET,POST,PUT,OPTIONS', 'Vary': 'Origin' };
  const out = (code, o) => { res.writeHead(code, Object.assign({ 'Content-Type':
'application/json', 'Cache-Control': 'no-store', 'X-Content-Type-Options': 'nosniff'
}, cors)); res.end(JSON.stringify(o)); };
  if (req.method === 'OPTIONS') { res.writeHead(204, cors); return res.end(); }
  const ip = String(req.headers['x-forwarded-for'] || req.socket.remoteAddress ||
```

```
'').split(',')[0].trim();
  try {
    if (limited(ip, 'g', 240, 60000)) throw E(429, 'Too many requests');
    const body = req.method === 'GET' ? {} : await readJson(req), m = req.method + '
' + url, now = Date.now();
    const authed = () => { const u = acctFromToken((req.headers.authorization ||
'').replace(/^Bearer /, '')); if (!u) throw E(401, 'invalid session'); if (u.blocked)
throw E(403, 'account blocked'); return u; };
    const need = (u, ...r) => { if (!r.includes(u.role)) throw E(403, 'forbidden');
};
    const target = () => { const t = DB.users.find(x => x.pid ===
String(body.playerId || '')); if (!t) throw E(404, 'player not found'); return t; };
    if (m === 'POST /api/register') {
      if (limited(ip, 'a', 20, 15 * 60000)) throw E(429, 'Too many attempts, try
later');
      const { username, password, inviteCode, setupCode } = body;
      if (!okName(username)) throw E(400, 'Username: 3-20 letters, digits,
underscore'); if (!okPass(password)) throw E(400, 'Password must be 8+ characters');
      if (DB.users.some(u => u.username.toLowerCase() === username.toLowerCase()))
throw E(409, 'Username taken');
      let inv = null; if (inviteCode) { inv = DB.invites.find(i => i.code ===
String(inviteCode).toUpperCase()); if (!inv || inv.revoked || inv.expires < now ||
inv.uses >= inv.max) throw E(400, 'Invite invalid, expired or used up'); }
      let role = 'player', om = false;   // the FIRST owner is created once with the
setup code; nobody can claim it by username or by editing JS
      if (setupCode) { const want = process.env.OWNER_SETUP_CODE || DB.setup; const a
= Buffer.from(String(setupCode)), b = Buffer.from(String(want || ''));
        if (hasOwner() || !want || a.length !== b.length ||
!crypto.timingSafeEqual(a, b)) throw E(403, 'Setup code rejected'); role = 'owner';
om = true; }
      const u = { id: DB.seq++, pid: 'ND-' +
crypto.randomBytes(4).toString('hex').toUpperCase(), username, pass:
hashPw(password), role, ownerMode: om, blocked: false, tv: 0, avatar: '🙂', seen:
now, created: now };
      DB.users.push(u); if (role === 'owner') { DB.setup = ''; audit('system',
username, 'owner-bootstrap'); }
      if (inv) { inv.uses++; DB.friends.push({ a: inv.inviter, b: u.id, blocked:
false }, { a: u.id, b: inv.inviter, blocked: false }); }
      dbSave(); return out(200, { token: signTok(u), me: viewU(u) });
    }
    if (m === 'POST /api/login') {
      if (limited(ip, 'a', 20, 15 * 60000)) throw E(429, 'Too many attempts, try
later');
      const u = typeof body.username === 'string' && DB.users.find(x =>
x.username.toLowerCase() === body.username.toLowerCase());
      if (!u || typeof body.password !== 'string' || !checkPw(body.password, u.pass))
throw E(401, 'Wrong username or password'); if (u.blocked) throw E(403, 'account
blocked');
      return out(200, { token: signTok(u), me: viewU(u) });
    }
```

```
    const u = authed();
    if (m === 'POST /api/logout') { u.tv++; dbSave(); return out(200, { ok: 1 }); }
    if (m === 'GET /api/me') return out(200, { me: viewU(u) });
    if (m === 'POST /api/ping') { u.seen = now; return out(200, { ok: 1 }); }
    if (m === 'PUT /api/profile') { u.avatar = String(body.avatar || '').slice(0, 8)
|| '🙂'; dbSave(); return out(200, { ok: 1 }); }
    if (m === 'GET /api/save') return out(200, DB.saves[u.id] || { data: null });
    if (m === 'PUT /api/save') { if (typeof body.data !== 'string' ||
body.data.length > 900000) throw E(400, 'bad save'); try { JSON.parse(body.data); }
catch (e) { throw E(400, 'bad save'); }
      DB.saves[u.id] = { data: body.data, perm: typeof body.perm === 'string' ?
body.perm : null, updated: now }; dbSave(); return out(200, { ok: 1 }); }
    if (m === 'POST /api/invites') { if (DB.invites.filter(i => i.inviter === u.id &&
!i.revoked && i.expires > now).length >= 10) throw E(429, 'Too many active invites');
      const code = crypto.randomBytes(5).toString('hex').toUpperCase();
DB.invites.push({ code, inviter: u.id, uses: 0, max: 5, expires: now + 7 * 864e5,
revoked: false }); dbSave(); return out(200, { code }); }
    if (m === 'GET /api/invites') return out(200, { invites: DB.invites.filter(i =>
i.inviter === u.id).slice(-20).map(i => ({ code: i.code, uses: i.uses, maxUses:
i.max, revoked: i.revoked })) });
    let r; if ((r = /^POST \/api\/invites\/([A-Za-z0-9]+)\/revoke$/.exec(m))) { const
i = DB.invites.find(x => x.code === r[1].toUpperCase() && x.inviter === u.id); if (i)
i.revoked = true; dbSave(); return out(200, { ok: 1 }); }
    if (m === 'GET /api/friends') return out(200, { friends: DB.friends.filter(f =>
f.a === u.id && !f.blocked).map(f => DB.users.find(x => x.id === f.b)).filter(x => x
&& !x.blocked).map(x => ({ playerId: x.pid, username: x.username, avatar: x.avatar,
online: isOnline(x) })) });   // never phone numbers / private data
    if ((r = /^POST \/api\/friends\/([A-Za-z0-9-]+)\/block$/.exec(m))) { const t =
DB.users.find(x => x.pid === r[1]); if (!t) throw E(404, 'not found');
      [[u.id, t.id], [t.id, u.id]].forEach(([a, b]) => { const f = DB.friends.find(x
=> x.a === a && x.b === b); if (f) f.blocked = true; else if (a === u.id)
DB.friends.push({ a, b, blocked: true }); }); dbSave(); return out(200, { ok: 1 }); }
    if (m === 'GET /api/admin/players') { need(u, 'owner', 'admin'); return out(200,
{ players: DB.users.slice(0, 500).map(x => ({ playerId: x.pid, username: x.username,
role: x.role, ownerMode: !!x.ownerMode, blocked: !!x.blocked, online: isOnline(x) }))
}); }
    if (m === 'POST /api/admin/role') { need(u, 'owner'); const t = target(); if (!
['admin', 'player'].includes(body.role)) throw E(400, 'role must be admin or
player'); if (t.role === 'owner') throw E(403, 'Owner cannot be changed');
      audit(u.username, t.username, 'role', t.role + ' → ' + body.role); t.role =
body.role; t.tv++; dbSave(); return out(200, { ok: 1 }); }
    if (m === 'POST /api/admin/perm') { need(u, 'owner'); const t = target(); if
(t.role === 'owner') throw E(403, 'Owner cannot be changed'); t.ownerMode =
body.ownerMode === true; audit(u.username, t.username, t.ownerMode ? 'grant
ownerMode' : 'revoke ownerMode'); dbSave(); return out(200, { ok: 1 }); }
    if (m === 'POST /api/admin/block') { need(u, 'owner', 'admin'); const t =
target(); if (t.role === 'owner' || (t.role === 'admin' && u.role !== 'owner') ||
t.id === u.id) throw E(403, 'forbidden');
      t.blocked = body.blocked === true; t.tv++; audit(u.username, t.username,
t.blocked ? 'block' : 'unblock');
```

```
      if (t.blocked) sessions.forEach(s => { if (s.acct === t.id && s.ws) { send(s, {
t: 'err', msg: 'Disconnected: account blocked' }); removePlayer(s, 'kicked'); try {
s.ws.close(1008); } catch (e) { /* ignore */ } } });
      dbSave(); return out(200, { ok: 1 }); }
    if (m === 'GET /api/admin/audit') { need(u, 'owner', 'admin'); return out(200, {
log: DB.audit.slice(-100).reverse() }); }
    throw E(404, 'not found');
  } catch (e) { out(e.code || 400, { error: e.code ? e.message : 'bad request' }); }
}

/* ---------------------------------------------------------------- HTTP */
const server = http.createServer((req, res) => {
  const url = req.url.split('?')[0];
  if (url.startsWith('/api/')) return api(req, res, url);
  if (url === '/health') { res.writeHead(200, { 'Content-Type': 'text/plain' });
return res.end('ok ' + rooms.size + ' rooms'); }
  if (url === '/' || url === '/index.html') {
    fs.readFile(GAME_FILE, (err, data) => {
      if (err) { res.writeHead(500, { 'Content-Type': 'text/plain' }); return
res.end('index.html not found next to server.js'); }
      res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8', 'Cache-
Control': 'no-cache', 'X-Content-Type-Options': 'nosniff' });
      res.end(data);
    });
    return;
  }
  res.writeHead(404, { 'Content-Type': 'text/plain' }); res.end('Not found');
});
server.on('upgrade', (req, socket) => {
  const url = req.url.split('?')[0];
  const key = req.headers['sec-websocket-key'];
  if (url !== '/ws' || !key || String(req.headers.upgrade || '').toLowerCase() !==
'websocket') { socket.destroy(); return; }
  if (ALLOWED_ORIGINS.length && !ALLOWED_ORIGINS.includes(req.headers.origin || ''))
{ socket.write('HTTP/1.1 403 Forbidden\r\n\r\n'); socket.destroy(); return; }
  const accept = crypto.createHash('sha1').update(key + WS_GUID).digest('base64');
  socket.write('HTTP/1.1 101 Switching Protocols\r\nUpgrade: websocket\r\nConnection:
Upgrade\r\nSec-WebSocket-Accept: ' + accept + '\r\n\r\n');
  onConnection(new WS(socket));
});

/* ---------------------------------------------------------------- state */
const rooms = new Map();     // code -> room
const sessions = new Map();  // pid -> player

function rid(n) { const a = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789'; let s = ''; for (let
i = 0; i < n; i++) s += a[crypto.randomInt(a.length)]; return s; }
function newCode() { for (;;) { const c = String(100000 + crypto.randomInt(900000));
if (!rooms.has(c)) return c; } }
function cleanName(n) { n = String(n == null ? '' : n).replace(/[^\p{L}\p{N}
```

```
_.\-]/gu, '').replace(/\s+/g, ' ').trim().slice(0, 14); return n || ('Player' +
crypto.randomInt(100, 999)); }
function num(v, lo, hi, d) { v = Number(v); return Number.isFinite(v) ? Math.max(lo,
Math.min(hi, v)) : d; }
function send(p, o) { if (p && p.ws && p.connected) p.ws.send(JSON.stringify(o)); }
function bcast(room, o, except) { const s = JSON.stringify(o); room.players.forEach(p
=> { if (p.connected && p.ws && p !== except) p.ws.send(s); }); }

function roomInfo(room) {
  const c = modeCfg(room.mode);
  return {
    t: 'room', code: room.code, host: room.host, mode: room.mode, modeName: c.name,
max: room.max, state: room.state,
    seed: room.seed, goal: c.goal, cp: c.cp, dur: c.dur, min: c.min, startAt:
room.startAt, now: Date.now(),
    players: [...room.players.values()].map(p => ({ id: p.id, name: p.name, color:
p.color, ready: p.ready, host: p.id === room.host, slot: p.slot, connected:
p.connected, fin: p.fin ? p.fin.place : 0, score: p.score, cp: p.cp }))
  };
}
function pushRoom(room) { bcast(room, roomInfo(room)); }

function freeColor(room) { const used = new Set([...room.players.values()].map(p =>
p.color)); for (let i = 0; i < 8; i++) if (!used.has(i)) return i; return 0; }
function freeSlot(room) { const used = new Set([...room.players.values()].map(p =>
p.slot)); for (let i = 0; i < 8; i++) if (!used.has(i)) return i; return 0; }
function slotTrk(slot) { return 34 - Math.floor(slot / 2) * 7; }

function resetProgress(room) {
  room.finCount = 0; room.firstFinAt = 0; room.startAt = 0; room.endAt = 0;
room.results = null;
  room.players.forEach(p => { p.st = null; p.cp = 0; p.fin = null; p.score = 0;
p.strikes = []; p.lastEv = 0; p.ready = false; p.base = slotTrk(p.slot); });
}

function addPlayer(room, p) {
  p.room = room; p.color = freeColor(room); p.slot = freeSlot(room); p.ready = false;
p.st = null; p.cp = 0; p.fin = null; p.score = 0; p.strikes = []; p.lastEv = 0;
p.base = slotTrk(p.slot);
  room.players.set(p.id, p);
}
function removePlayer(p, why) {
  const room = p.room; if (!room) return;
  room.players.delete(p.id); p.room = null;
  bcast(room, { t: 'left', id: p.id, why: why || 'left' });
  if (room.players.size === 0) { clearTimeout(room.cdTimer); rooms.delete(room.code);
return; }
  if (room.host === p.id) {
    const nh = [...room.players.values()].find(q => q.connected) ||
[...room.players.values()][0];
```

```
    room.host = nh.id;
  }
  if (room.state === 'countdown' && connectedCount(room) < modeCfg(room.mode).min) {
clearTimeout(room.cdTimer); room.state = 'lobby'; resetProgress(room); }
  pushRoom(room);
  if (room.state === 'racing') checkEnd(room);
}
function connectedCount(room) { let n = 0; room.players.forEach(p => { if
(p.connected) n++; }); return n; }

/* ------------------------------------------------------------------ connection */
function onConnection(ws) {
  let p = null;
  ws.onmessage = raw => {
    const now = Date.now();
    if (now - ws.bucket.t > 1000) { ws.bucket.t = now; ws.bucket.n = 0; }
    if (++ws.bucket.n > 70) { if (ws.bucket.n > 200) ws.close(1008); return; }
    if (raw.length > 2048) return;
    let m; try { m = JSON.parse(raw); } catch (e) { return; }
    if (!m || typeof m.t !== 'string') return;
    if (m.t === 'hello') {   // optional account token. Guests are allowed (as
before) unless REQUIRE_ACCOUNT=1, then only valid, unblocked accounts can play online
      const a = m.auth ? acctFromToken(m.auth) : null;
      if (REQUIRE_ACCOUNT && (!a || a.blocked)) { ws.send(JSON.stringify({ t: 'err',
msg: 'Disconnected: please log in with an account to play online' }));
ws.close(1008); return; }
      p = hello(ws, m); if (p && a) p.acct = a.id; return;
    }
    if (!p) return;
    handle(p, m);
  };
  ws.onclose = () => {
    if (!p || p.ws !== ws) return;
    p.connected = false; p.dcAt = Date.now();
    if (p.room) pushRoom(p.room); else sessions.delete(p.id);
  };
  ws.hello = setTimeout(() => { if (!p) ws.close(1008); }, 8000);
}

function hello(ws, m) {
  clearTimeout(ws.hello);
  let p = null;
  if (typeof m.pid === 'string' && sessions.has(m.pid)) {
    const s = sessions.get(m.pid);
    if (s.token === m.token) p = s;
  }
  if (p) {
    if (p.ws && p.ws !== ws) { const old = p.ws; p.ws = null; try { old.close(1000);
} catch (e) { /* ignore */ } }
    p.ws = ws; p.connected = true; p.dcAt = 0;
```

```
    if (m.name) p.name = cleanName(m.name);
    send(p, { t: 'welcome', pid: p.id, token: p.token, name: p.name, resumed: true,
now: Date.now() });
    if (p.room) pushRoom(p.room);
  } else {
    p = { id: rid(6), token: crypto.randomBytes(12).toString('hex'), name:
cleanName(m.name), ws, connected: true, dcAt: 0, room: null };
    sessions.set(p.id, p);
    send(p, { t: 'welcome', pid: p.id, token: p.token, name: p.name, resumed: false,
now: Date.now() });
  }
  return p;
}

function err(p, msg) { send(p, { t: 'err', msg }); }

function handle(p, m) {
  const room = p.room;
  switch (m.t) {
    case 'ping': send(p, { t: 'pong', c: m.c, ts: Date.now() }); return;
    case 'create': {
      if (room) return err(p, 'Already in a room');
      const mode = MODES[m.mode] ? m.mode : 'race';
      const r = { code: newCode(), host: p.id, mode, max: Math.round(num(m.max, 2, 8,
8)), state: 'lobby', seed: crypto.randomInt(1, 2147483647), players: new Map(),
startAt: 0, endAt: 0, finCount: 0, firstFinAt: 0, cdTimer: null, results: null,
created: Date.now() };
      rooms.set(r.code, r); addPlayer(r, p); pushRoom(r); return;
    }
    case 'join': {
      if (room) return err(p, 'Already in a room');
      const code = String(m.code || '').replace(/\D/g, '').slice(0, 6);
      const r = rooms.get(code);
      if (!r) return err(p, 'Room not found');
      if (r.players.size >= r.max) return err(p, 'Room is full');
      if (r.state !== 'lobby' && !(r.mode === 'roam' && r.state === 'racing')) return
err(p, 'Race already started');
      addPlayer(r, p);
      if (r.state === 'racing') { p.ready = true; p.base = avgTrk(r) || p.base; }
      pushRoom(r);
      if (r.state === 'racing') send(p, { t: 'go', startAt: r.startAt, late: true,
base: p.base });
      return;
    }
    case 'leave': removePlayer(p, 'left'); return;
    case 'ready': if (room && room.state === 'lobby') { p.ready = !!m.v;
pushRoom(room); } return;
    case 'cfg': {
      if (!room || room.host !== p.id || room.state !== 'lobby') return;
      if (MODES[m.mode]) room.mode = m.mode;
```

```
      if (m.max != null) room.max = Math.max(room.players.size, Math.round(num(m.max,
2, 8, room.max)));
      room.players.forEach(q => { q.ready = false; });
      pushRoom(room); return;
    }
    case 'start': startRoom(p); return;
    case 'end': if (room && room.host === p.id && room.state !== 'lobby')
toLobby(room); return;
    case 's': onState(p, m); return;
    case 'ev': onEvent(p, m); return;
  }
}

function avgTrk(room) { let s = 0, n = 0; room.players.forEach(q => { if (q.st) { s
+= q.st.trk; n++; } }); return n ? s / n : 0; }

function startRoom(p) {
  const room = p.room; if (!room || room.host !== p.id || room.state !== 'lobby')
return;
  const c = modeCfg(room.mode);
  if (connectedCount(room) < c.min) return err(p, 'Need at least ' + c.min + '
player' + (c.min > 1 ? 's' : '') + ' for this mode');
  for (const q of room.players.values()) if (q.connected && q.id !== room.host &&
!q.ready) return err(p, q.name + ' is not ready');
  resetProgress(room);
  room.players.forEach(q => { q.ready = true; });
  if (room.mode === 'roam') return beginRace(room);
  room.state = 'countdown'; let n = 3;
  pushRoom(room); bcast(room, { t: 'cd', n });
  room.cdTimer = setInterval(() => {
    n--;
    if (n > 0) bcast(room, { t: 'cd', n });
    else { clearInterval(room.cdTimer); beginRace(room); }
  }, 1000);
}
function beginRace(room) {
  room.state = 'racing'; room.startAt = Date.now();
  const c = modeCfg(room.mode); if (c.dur) room.endAt = room.startAt + c.dur * 1000;
  room.players.forEach(q => { q.lastAt = room.startAt; });
  pushRoom(room); bcast(room, { t: 'go', startAt: room.startAt });
}
function toLobby(room) {
  clearInterval(room.cdTimer); room.state = 'lobby'; resetProgress(room);
pushRoom(room);
}

/* ---------------------------------------------------------------- validation */
function strike(p, why) {
  const now = Date.now(); p.strikes.push(now); p.strikes = p.strikes.filter(t => now
- t < 10000);
```

```
  if (p.strikes.length >= 10) { send(p, { t: 'err', msg: 'Disconnected: invalid
movement (' + why + ')' }); removePlayer(p, 'kicked'); try { p.ws.close(1008); }
catch (e) { /* ignore */ } return true; }
  return false;
}
function correct(p) { const s = p.st; send(p, { t: 'corr', trk: s ? s.trk : p.base,
px: s ? s.px : 0 }); }

function onState(p, m) {
  const room = p.room; if (!room || (room.state !== 'racing' && room.state !==
'countdown')) return;
  const a = m.a; if (!Array.isArray(a) || a.length < 9) return;
  const v = a.slice(0, 9).map(Number);
  if (!v.every(Number.isFinite)) { strike(p, 'nan'); return; }
  const [trk, px, psi, spd, steer, fl, y, roll, pitch] = v;
  if (Math.abs(px) > 14 || Math.abs(psi) > 3.2 || Math.abs(spd) > VMAX ||
Math.abs(steer) > 1.2 || y < -2 || y > 45 || Math.abs(roll) > 20 || Math.abs(pitch) >
20 || trk < -5 || trk > 1e7) { if (!strike(p, 'range')) correct(p); return; }
  const now = Date.now();
  if (!p.st) {
    if (Math.abs(trk - p.base) > 60) { if (!strike(p, 'start')) correct(p); return; }
  } else {
    const dt = Math.min(3, Math.max(0.05, (now - p.lastAt) / 1000));
    const lim = VMAX * 1.3 * dt + 3;
    if (Math.abs(trk - p.st.trk) > lim) { if (!strike(p, 'teleport')) correct(p);
return; }
    if (room.state === 'countdown' && Math.abs(trk - p.base) > 4) { if (!strike(p,
'falsestart')) correct(p); return; }
  }
  const prev = p.st ? p.st.trk : trk;
  p.st = { trk, px, psi, spd, steer, fl: fl | 0, y, roll, pitch }; p.lastAt = now;
  if (room.state !== 'racing') return;
  const c = modeCfg(room.mode);
  if (c.cp) {
    const ncp = Math.floor(c.goal / c.cp);
    const nextS = (p.cp + 1) * c.cp;
    if (p.cp < ncp && prev < nextS && trk >= nextS && Math.abs(px) <= 7.8) {
      p.cp++; bcast(room, { t: 'cp', id: p.id, i: p.cp, n: ncp, time: (now -
room.startAt) / 1000 });
    }
  }
  if (c.goal && !p.fin && trk >= c.goal) {
    const need = room.mode === 'checkpoint' ? Math.floor(c.goal / c.cp) : 0;
    if (p.cp >= need) finish(room, p, now);
  }
}
function finish(room, p, now) {
  p.fin = { place: ++room.finCount, time: (now - room.startAt) / 1000 };
  if (!room.firstFinAt) room.firstFinAt = now;
  bcast(room, { t: 'fin', id: p.id, place: p.fin.place, time: p.fin.time });
```

```
  pushRoom(room); checkEnd(room);
}
function onEvent(p, m) {
  const room = p.room; if (!room || room.state !== 'racing' || room.mode !== 'stunt')
return;
  const now = Date.now(); if (now - p.lastEv < 600) return;
  const pts = Math.round(num(m.pts, 0, 8000, 0)); if (pts <= 0) return;
  const elapsed = (now - room.startAt) / 1000;
  if (p.score + pts > 4000 + elapsed * 1800) { strike(p, 'score'); return; }
  p.lastEv = now; p.score += pts;
}

/* ------------------------------------------------------------------ race end +
results */
function checkEnd(room) {
  if (room.state !== 'racing') return;
  const c = modeCfg(room.mode), now = Date.now();
  if (room.mode === 'roam') return;
  if (room.mode === 'stunt') { if (now >= room.endAt) endRace(room); return; }
  let active = 0, unfinished = 0;
  room.players.forEach(q => { if (q.connected) { active++; if (!q.fin) unfinished++;
} });
  if (active === 0 || unfinished === 0) return endRace(room);
  if (room.firstFinAt && now - room.firstFinAt > 25000 && room.mode !== 'trial')
return endRace(room);
  if (now - room.startAt > 8 * 60000) endRace(room);
}
function endRace(room) {
  if (room.state !== 'racing') return;
  room.state = 'finished';
  const c = modeCfg(room.mode), list = [...room.players.values()];
  if (room.mode === 'stunt') list.sort((a, b) => b.score - a.score);
  else list.sort((a, b) => (a.fin ? 0 : 1) - (b.fin ? 0 : 1) || (a.fin && b.fin ?
a.fin.place - b.fin.place : ((b.st ? b.st.trk : 0) - (a.st ? a.st.trk : 0))));
  const out = list.map((q, i) => {
    const place = i + 1, done = room.mode === 'stunt' ? true : !!q.fin;
    const xp = done ? (XP_BY_PLACE[i] || 30) : 20;
    return { id: q.id, name: q.name, color: q.color, place, time: q.fin ? q.fin.time
: null, score: q.score, dnf: !done, xp, cash: xp * 4, coins: Math.round(xp / 5) +
(room.mode === 'stunt' ? Math.round(q.score / 200) : 0) };
  });
  room.results = out;
  bcast(room, { t: 'results', mode: room.mode, list: out });
  pushRoom(room);
  clearTimeout(room.backTimer);
  room.backTimer = setTimeout(() => { if (room.state === 'finished' &&
rooms.has(room.code)) toLobby(room); }, 30000);
}

/* ------------------------------------------------------------------ loops */
```

```
setInterval(() => {
  const now = Date.now();
  rooms.forEach(room => {
    if (room.state === 'racing' || room.state === 'countdown') {
      const arr = [];
      room.players.forEach(q => { if (q.st) { const s = q.st; arr.push([q.id,
+s.trk.toFixed(2), +s.px.toFixed(2), +s.psi.toFixed(3), +s.spd.toFixed(1),
+s.steer.toFixed(3), s.fl, +s.y.toFixed(2), +s.roll.toFixed(2), +s.pitch.toFixed(2),
q.cp, q.score, q.fin ? q.fin.place : 0]); } });
      if (arr.length) bcast(room, { t: 'snap', ts: now, p: arr });
      if (room.state === 'racing') checkEnd(room);
    }
    room.players.forEach(q => { if (!q.connected && q.dcAt && now - q.dcAt >
GRACE_MS) { sessions.delete(q.id); removePlayer(q, 'timeout'); } });
  });
  sessions.forEach((q, id) => { if (!q.room && !q.connected && now - (q.dcAt || now)
> GRACE_MS) sessions.delete(id); });
}, TICK_MS);
setInterval(() => {   // heartbeat
  const now = Date.now();
  sessions.forEach(q => { if (q.ws && q.connected) { if (now - q.ws.lastSeen > 45000)
q.ws.close(1001); else q.ws.ping(); } });
}, 15000);

server.listen(PORT, HOST, () => {
  console.log('Night Drive 3D server listening on http://' + (HOST === '0.0.0.0' ?
'localhost' : HOST) + ':' + PORT);
  console.log('WebSocket endpoint: /ws   game file: ' + GAME_FILE);
  console.log('Accounts API: /api/*   data folder: ' + DATA_DIR +
(process.env.DATA_DIR ? '' : '  (set DATA_DIR to a persistent disk when hosting)'));
  if (!hasOwner()) console.log('FIRST OWNER SETUP CODE (use once on the Account
screen, then it is deleted): ' + (process.env.OWNER_SETUP_CODE || DB.setup));
});
process.on('uncaughtException', e => console.error('uncaught', e));
```